CS 3892 / CS 5892 — Special Topics

Formal Methods: Safe and Trustworthy Artificial Intelligence

Vanderbilt University · Fall 2026 · 3 credits
Tuesday / Thursday, 11:00 a.m. – 12:15 p.m.

> **Draft — pre-semester.** Items marked **[confirm]** are pending institutional confirmation (cross-listing, room, TA assignment, Immersion Showcase date). Everything else is final unless the registrar changes the academic calendar.

---

1. Instructor and course staff

Instructor: Taylor T. Johnson, PhD, PE
Email: taylor.johnson@vanderbilt.edu · Web: https://www.taylortjohnson.com

Office hours: by appointment — email to schedule. Easiest immediately before or after class; my office is in ISIS.

Teaching assistant: TBA **[confirm]** · TA office hours: TBA

Course materials and announcements: **Brightspace**. Code, assignment starters, and slides: the course GitHub repository (link on Brightspace).

2. Course description

This project course studies formal methods for establishing the trustworthiness of systems, with a focus on projects for safe and trustworthy artificial intelligence. Formal methods are mathematically rigorous techniques and tools for specifying, designing, and analyzing systems so that they demonstrably meet their requirements. We study model checking, satisfiability (SAT/SMT), reachability, abstract interpretation, automated and interactive theorem proving, and neural network verification — and, throughout, how each of these is being combined with AI systems, including large language models.

The subject has two directions and we take both seriously:

- **FM4AI** — using formal methods *on* AI systems. Verifying robustness of a neural network; proving that an agent cannot exfiltrate a secret; checking an LLM's answer against a policy with a solver rather than another model.
- **AI4FM** — using AI *to do* formal methods. Autoformalizing English requirements into temporal logic; LLM-assisted proof search in Lean; generating specifications, invariants, and counterexample explanations.

Students apply these methods in a semester-long team project. The course counts as the **CS project requirement** and may be used to satisfy Vanderbilt's **Immersion** requirement.

Prerequisites (topical): elementary discrete mathematics — propositional, first-order, and higher-order logic; set theory; Boolean operations; basic graph theory — and elementary programs and algorithms, including syntax and semantics. You should be able to write, compile, run, and debug programs in more than one language, and be comfortable at a command line. Familiarity with AI and machine learning, including generative AI and LLMs, is assumed.

Recommended course prerequisites: CS 3251 and CS 3270, plus one of CS 3262, CS 4260, CS 4262, CS 4267, or ECE 4363.

Cross-listing: CS 3892 (undergraduate) and CS 5892 (graduate) **[confirm]**. Graduate students complete additional project components — see §7.

Why this course, and why now

Generative AI has industrialized software *production* without a matching advance in software *validation*. The gap is not hypothetical. In the twelve months before this course begins, an AI-orchestrated cyber-espionage campaign ran with 80–90% of its tactical work unsupervised; a coding agent destroyed a production database, backups included, in about nine seconds; Amazon required senior sign-off on AI-assisted changes after outages its own internal briefings tied to "Gen-AI assisted changes" with "high blast radius"; and two U.S. federal judges withdrew opinions containing hallucinated citations.

Formal methods are the part of computer science that produces *guarantees* rather than test results. This course asks what those guarantees can and cannot cover when the artifact under scrutiny is a neural network or an agent — and, symmetrically, whether AI can finally make formal methods cheap enough to use routinely.

3. Learning objectives

By the end of the course you will be able to:

1. Explain what formal methods are, what they guarantee, and — equally important — what they do not, and where they sit in a software and systems engineering process.
2. Describe AI risks and failure modes — hallucination, jailbreaking, adversarial perturbation, backdoors, prompt injection, tool-use and agentic failures, IP and provenance — and analyze real incidents against a risk taxonomy.
3. Translate informal requirements into formal specifications: invariants, pre/postconditions, and temporal logic; and assess when each formalism is expressive enough and when it is too costly.
4. Model a reactive system as a transition system and verify safety and liveness properties with a model checker, reading and explaining counterexample traces.
5. Encode decision problems for SAT and SMT solvers and explain how these engines underpin much of the field, including industrial hallucination-prevention systems.
6. Apply static analysis, dynamic analysis, and abstract interpretation to programs, and explain the precision/cost trade-off a choice of abstract domain buys.
7. Write and complete machine-checked proofs in **Lean 4**, including proving an invariant of a transition system by the inductive-invariant method, and relate the automata-theoretic foundations of model checking to their formalization in **Cslib**.
8. Verify properties of neural networks, interpret verified / falsified / timeout outcomes correctly, and state honestly where the scaling limits lie.
9. Use generative AI as an instrument of verification — autoformalization, proof search, spec generation — and *evaluate its output critically*, including detecting confident, plausible, and wrong results.

10. Execute a substantial team project and communicate it in writing, in a talk, and in a demo.

4. Course format and calendar

Two 75-minute meetings per week, Tuesday and Thursday, 11:00 a.m.–12:15 p.m. Room TBA **[confirm]**. Sessions mix lecture, live tool demonstrations, in-class case-study discussion, and project checkpoints. **Bring a laptop.**

Key Fall 2026 dates (Vanderbilt Registrar, 2026–27 academic calendar; calendars are subject to change):

Date	Event
Wed Aug 26	First day of classes (our first meeting is Thu Aug 27)
Fri Sep 4	Last day to add/drop
Thu–Fri Oct 22–23	Fall Break — no class Thu Oct 22
Fri Oct 30	Last day to withdraw; last day for pass/fail election
Sun Nov 22 – Sun Nov 29	Thanksgiving — no classes Nov 24 or Nov 26
Thu Dec 10	Last day of classes
Thu Dec 10, 5–7 p.m.	**CS Immersion Showcase**, Adams Atrium, Featheringill Hall **[confirm]**
Fri Dec 11, Sun Dec 13	Reading days
Sat Dec 12, Mon Dec 14 – Sat Dec 19	Final examination period

There is **no final exam**. The final project presentation and deliverables take its place.

> **Plan around this:** the Immersion Showcase falls on the last day of classes. If you are using this course for the CS project or Immersion requirement, your poster or demo must be finished *before* Dec 10, not on it.

The full session-by-session schedule, with all homework and project dates, is in `schedule/schedule.md`.

5. Topics

Unit 0 · Framing (session 1). What formal methods are and what they buy. The verification triple: model + specification → verifier → verdict. FM4AI and AI4FM.

Unit 1 · AI failure modes, risk, and governance (sessions 2–4). A taxonomy of failure: alignment, hallucination, toxicity, jailbreaking, adversarial perturbation, backdoors and trojans, data provenance, IP, human factors. Prompt injection and agentic tool-use failures. Case studies from the past year. Guardrails, red-teaming, model cards and datasheets. Risk frameworks and regulation — NIST AI RMF (and the fact that it is currently under revision), NIST AI 100-2 adversarial-ML taxonomy, the EU AI Act as amended in July 2026, ISO/IEC 24029-2, EASA's aviation AI concept paper. The central contrast of the course, introduced early: a *filter* that another model evaluates, versus a *check* that a solver decides.

Unit 2 · Logic and preliminaries (sessions 5–6). Sets and set-builder notation; predicates; propositional, first-order, and higher-order logic; satisfiability versus validity; the duality between formulas and sets of states.

Unit 3 · SAT, SMT, and decision procedures (sessions 7–8). DPLL/CDCL; theories (linear arithmetic, bit-vectors, uninterpreted functions); DPLL(T). Z3 from Python and SMT-LIB. Encoding problems as constraints. Bounded reachability. Why SMT is the engine under industrial hallucination checking.

Unit 4 · Modeling and specification (session 9). Transition systems `T = (S, S₀, →, AP, L)`; traces and reachability; invariants and *inductive* invariants; strengthening. The SMV language.

Unit 5 · Temporal logic (sessions 10, 12). LTL and CTL; safety versus liveness; ω-words and ω-regular languages; Büchi automata and the automata-theoretic view of model checking. Signal temporal logic and quantitative robustness. Temporal properties over *agent execution traces* — the specification problem that per-request content filtering cannot express.

Unit 6 · Model checking (sessions 13–14). Explicit-state, symbolic (BDD), and bounded (SAT) model checking, and when each is the right tool. Image computation, fixpoints, quantifier elimination. nuXmv in practice; counterexample-guided debugging; k-induction and IC3.

Unit 7 · Program analysis (sessions 15–17). Programs as transition systems over memory states. Bounded model checking of C with CBMC; loop unwinding and unwinding assertions. Deductive verification, Hoare logic, loop invariants, ACSL and Frama-C. Abstract interpretation: abstract domains, Galois connections, widening — and what precision is lost at a join. Static, dynamic, and concolic analysis; automated test generation.

Unit 8 · Interactive theorem proving (sessions 18–21). Lean 4: terms, types, `def`, `theorem`, tactics. Curry–Howard. Induction. Proving invariants of transition systems, and proving *parametric* statements a model checker cannot settle. Then **Cslib** — the Lean library for computer science — where the automata-theoretic foundations of Unit 5 are machine-checked: labelled transition systems, bisimulation, Hennessy–Milner logic and the Hennessy–Milner theorem, ω-languages, Büchi acceptance, the strict inclusion of deterministic in nondeterministic Büchi automata, and closure of ω-regular languages under complementation. We will also look carefully at what Cslib *does not* yet contain — there is no LTL, no LTL→Büchi translation, and no emptiness check — because the boundary between "proved" and "asserted on a slide" is precisely what this course is about.

Unit 9 · AI for formal methods (sessions 23–24). Autoformalization: English → temporal logic, English → SMV, English → ACSL, English → Lean. LLM proof search and the current state of AI theorem proving. What is genuinely solved, what is saturated benchmark theater, and the **soundness gap**: a machine-checked proof of a machine-generated formalization certifies only that the two agree. The Erdős #728 episode — where formalization in Lean is exactly what separated a real result from a retracted claim — is the argument for this entire course, delivered by current events.

Unit 10 · Neural network verification (sessions 25–26). Robustness as a formal property. Incomplete versus complete methods; interval bound propagation, CROWN-style linear relaxation, branch-and-bound. ONNX and VNN-LIB (and the 2.0 revision, which gives the format a formal semantics mechanized in Agda). Tools: α,β-CROWN, NNV, n2v, NeuralSAT, PyRAT, Marabou. VNN-COMP 2026 results as a live dataset. An honest account of the scaling limits, and of certification practice in aviation.

Unit 11 · Project presentations (sessions 27–28). Proposal talks are session 11 and the milestone is session 22.

6. Materials

No required textbook. Everything below is free, and everything is optional except where a reading is assigned in class.

Tools

Every tool has a **zero-install path**. You will not need to build anything locally to pass this course, though you are welcome to.

Tool	Use	Zero-install
Z3	SMT solving	Z3 Guide, in-browser: https://microsoft.github.io/z3guide/ · repo: https://github.com/Z3Prover/z3
nuXmv / NuSMV	Model checking	smvis, in-browser: https://smvis-378135919048.us-central1.run.app/ · nuXmv is licence-gated, so we use NuSMV where needed
Lean 4	Interactive theorem proving	https://live.lean-lang.org/ · Natural Number Game: https://adam.math.hhu.de/
Cslib	CS structures in Lean	https://github.com/leanprover/cslib — note the org is `leanprover`, **not** `leanprover-community`
CBMC	Bounded model checking of C	GitHub Codespaces (course container) · https://github.com/diffblue/cbmc
Frama-C	Deductive verification, ACSL	https://frama-c.com/
α,β-CROWN / auto_LiRPA	Neural network verification	Google Colab (course notebooks)
Course container	Everything preinstalled	GitHub Codespaces via https://github.com/ttj/fmaiv

Lean version pinning — read this before installing anything. Lean releases monthly and there will be two or three releases during the semester. For the whole term we pin **Lean v4.33.0 + Mathlib v4.33.0 + Cslib v4.33.0**. Do not follow Cslib's README, which points at `main` and currently drags in a release-candidate toolchain. Install by installing VS Code first, then the **Lean 4** extension, and letting the extension run its guided setup. Note that the in-browser Lean editors ship Mathlib only — `import Cslib` will not work there, so Cslib work happens in Codespaces or locally.

Lab spine

Much of the hands-on work is built on **FMAIV** — *Formal Methods & AI-Assisted Verification*, a short course with worked examples, notebooks, and a Codespaces container:

- Repository: https://github.com/ttj/fmaiv · Slides: https://ttj.github.io/fmaiv/

FMAIV threads a single bounded counter through Z3, nuXmv, Lean, CBMC, and Cryptol, verifying the *same* property five ways. The homeworks follow that thread. By the end you will have a concrete, earned sense of what "bounded" versus "unbounded," and "checked" versus "proved," actually mean.

AI risk, standards, and governance

- Microsoft, *Failure Modes in Machine Learning* — https://learn.microsoft.com/en-us/security/engineering/failure-modes-in-machine-learning
- **NIST AI RMF 1.0** (= NIST AI 100-1) — https://www.nist.gov/itl/ai-risk-management-framework · https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf *Note: NIST states the RMF is being revised under the July 2025 AI Action Plan. We read it as a live document, not a settled one.*
- **NIST AI 600-1**, Generative AI Profile — https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
- **NIST AI 100-2 E2025**, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations* — https://csrc.nist.gov/pubs/ai/100/2/e2025/final (use the corrected April 2025 PDF together with the June 2025 errata)
- **EU AI Act**, consolidated, and Regulation (EU) 2026/1744 (the "Digital Omnibus on AI," in force 27 July 2026) — https://eur-lex.europa.eu/eli/reg/2026/1744/oj/eng *What matters for us: Article 50 transparency obligations apply from 2 August 2026; the high-risk obligations moved to December 2027 and August 2028. Any source dating high-risk from August 2026 predates the amendment.*
- **ISO/IEC 24029-2:2023**, robustness of neural networks — methodology for the use of formal methods — https://www.iso.org/standard/79804.html. Related: ISO/IEC 42001:2023 (AI management systems); ISO/IEC 24029-3 is in development.
- **EASA** AI concept paper, Proposed Issue 03 (June 2026) — the current aviation position; AI touching DAL A or DAL B failure conditions remains outside accepted means of compliance.
- **DoD AI Assurance (AIA) Toolkit** (formerly the Responsible AI Toolkit) — https://www.tradewindai.com/aia-toolkit
- **AI Incident Database** — https://incidentdatabase.ai/ · **OECD AI Incidents Monitor** — https://oecd.ai/en/incidents · **AI Hallucination Cases Database** — https://www.damiencharlotin.com/hallucinations/

Industrial formal methods for AI

- **AWS Bedrock Automated Reasoning checks** — SMT-backed policy checking of model output, GA August 2025 — https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html
- **AWS Bedrock Guardrails** — https://aws.amazon.com/bedrock/guardrails/
- Werner Vogels interviews Byron Cook, *A chat with Byron Cook on automated reasoning and trust in AI systems*, February 2026 — the single best introductory reading for the course's thesis — https://www.allthingsdistributed.com/2026/02/a-chat-with-byron-cook-on-automated-reasoning-and-trust-in-ai-systems.html
- **Cedar Analysis** — an authorization-policy verifier whose symbolic compiler is written in Lean and discharges to CVC5, with soundness and completeness proved in Lean.
- **Dogwood** — a governance language adding past-time temporal conditions (`since`, `formerly`, `once`) over an agent's recent events, lowering to Cedar, with a trace-`replay` CLI — https://github.com/dogwood-policy/dogwood. We use it as the worked example of a specification a per-request filter cannot express.
- **Adversarial Robustness Toolbox** — https://github.com/Trusted-AI/adversarial-robustness-toolbox

Verification competitions and benchmarks

- **VNN-COMP** — https://vnn-comp.github.io/ · 2026 results: https://github.com/VNN-COMP/vnncomp2026_results · **VNN-LIB 2.0** — https://www.vnnlib.org/
- **SV-COMP** — https://sv-comp.sosy-lab.org/ · **SMT-COMP** — https://smt-comp.github.io/ · **SAT competition** — https://satcompetition.github.io/
- **ARCH** (cyber-physical systems) — https://cps-vo.org/group/ARCH/benchmarks

Reference books

- Aho & Ullman, *Foundations of Computer Science* — http://infolab.stanford.edu/~ullman/focs.html
- Alur, *Principles of Cyber-Physical Systems*, MIT Press
- Clarke, Grumberg & Peled, *Model Checking*, MIT Press
- Nipkow & Klein, *Concrete Semantics* — https://concrete-semantics.org/
- *Functional Programming in Lean* — https://lean-lang.org/functional_programming_in_lean/ (gentlest Lean entry point for CS students)
- *Theorem Proving in Lean 4* — https://lean-lang.org/theorem_proving_in_lean4/
- *The Hitchhiker's Guide to Logical Verification*, 2026 edition — https://github.com/lean-forward/logical_verification_2026
- nuXmv user manual — https://es-static.fbk.eu/tools/nuxmv/downloads/nuxmv-user-manual.pdf

7. Assignments and grading

Component	Weight
Term project	**50%**
Homeworks (5)	20%
Quizzes (5, lowest dropped)	10%
Case studies, attendance, and in-class participation	20%

The instructor reserves the right to adjust these percentages, and to add assignments, if class participation proves insufficient.

Term project — 50%

The centerpiece. Team-based (2–4 members; a team is **required** for students using this course for the CS project requirement). You choose from a menu of eight defined topics, or propose your own with approval.

Deliverable	Share of course grade	When
Proposal — report plus lightning talk	5%	Thu Oct 1
Milestone — presentation plus **running artifact**	10%	Thu Nov 12
Final — report, presentation, demo, artifacts	35%	Dec 8 / Dec 10 and finals period

Full instructions, the topic menu, and rubrics are in the course repository under `projects/`.

A change from previous years, stated plainly. Topics are now provided rather than open, and the milestone is a *gate*: by Nov 12 you must have a verifier that runs and produces output. Last year several teams reached November with models trained and nothing verified, and there is not enough semester left after that point to recover. Both changes exist to protect your grade, not to constrain your ambition — every topic on the menu has a genuinely open stretch goal.

Every project must contain both an FM half and an AI half. The most common milestone feedback in Fall 2025 was, verbatim, *"where is the AI?"* The topic menu is constructed so this cannot happen by accident.

Homeworks — 20%

Five homeworks, roughly biweekly, 4% each. One tool per homework: Z3, then nuXmv, then CBMC and abstract interpretation, then Lean 4, then neural network verification. Each has an AI component in which you use an assistant and then *check its work with the tool* — which is the one setting where you can actually find out whether it was right. See `assignments/README.md`.

Late policy: four late days for the semester, whole-day units, at most two per homework, no questions asked. After that, 20% per day. Late days do not apply to project deliverables, which are presented in class.

Quizzes — 10%

Five short online quizzes in Brightspace, 10–15 minutes, auto-graded, taken before the class that uses the material. **The lowest score is dropped.** They exist so that lecture can assume you have done the reading, and so you find out early — cheaply — when you have not.

Case studies, attendance, and participation — 20%

This is a discussion course as much as a lecture course. Each student leads or co-leads one in-class case study during the semester: an AI failure, its mechanism, and what formal method (if any) would have caught it — including the honest answer "none, and here is why." Participation also covers in-class exercises, tool walkthroughs, and peer feedback on project presentations.

Grading scale

A ≥ 93 · A− ≥ 90 · B+ ≥ 87 · B ≥ 83 · B− ≥ 80 · C+ ≥ 77 · C ≥ 73 · C− ≥ 70 · D+ ≥ 67 · D ≥ 63 · D− ≥ 60 · F < 60. The instructor may curve upward; never downward.

Graduate students (CS 5892)

Graduate students are held to a higher standard throughout, and carry additional project components: a related-work survey of 8–12 papers that makes an argument rather than a list; a second independent evaluation axis (a second dataset, tool family, or ablation); an explicit threats-to-validity analysis with at least one experiment testing it; and a final report written to the standard of a workshop submission. In mixed teams, graduate members own these components, stated in the responsibility breakdown.

8. Attendance

Attendance is expected. If you cannot attend, tell me in advance — much of the value of the course is in the in-class activities and discussions, and they cannot be made up asynchronously. Attendance on project presentation days (Oct 1, Nov 12, Dec 8, Dec 10) is required; peer feedback on those days is part of the participation grade.

9. Generative AI policy

Generative AI is permitted on all assignments and projects, and its use is expected. This is a course about the trustworthiness of AI-produced artifacts; declining to use the technology would be beside the point. Two conditions:

1. **Disclose it.** Every homework and every project deliverable has an AI-use section. Say what you used and for what. Nobody is penalized for using AI; failing to disclose is an honour-code matter.
2. **You are accountable for what you submit.** "The model produced it" is not a defence. In this course it is a particularly weak one, because for nearly everything we do there is a decision procedure available that will tell you whether the model was right. Run it.

Submitting an unverified AI-generated proof is both an integrity problem and a strategic error: Lean will not accept `sorry`, and the compiler is a more rigorous grader than I am.

Vanderbilt guidance: https://www.vanderbilt.edu/generative-ai/students/ · https://www.vanderbilt.edu/generative-ai/academic-integrity/

10. Honor Code

As a Vanderbilt student it is your responsibility to read and understand the Honor Code: https://studenthandbook.vanderbilt.edu/community-standards

Collaboration and discussion are encouraged throughout. Submitted work must be your own, and external sources — including AI assistants — must be cited.

11. University policies and student support

Students with disabilities. Vanderbilt is committed to equal opportunity for students with disabilities. If you have a physical or learning disability, contact Student Access, which will notify your instructors so that appropriate accommodations can be arranged. Absent notification it is assumed no accommodations are sought. https://www.vanderbilt.edu/student-access/

If you learn best in a particular way, tell me — I would rather adjust than have you struggle, and I can only do that if I know in advance of the assignment. I will keep your needs confidential.

Audio/video recording and intellectual property. Students may not record, reproduce, screenshot, photograph, or distribute any content from their courses, including recordings of in-person or Zoom sessions, except for their own personal educational use. Instructors likewise may not generally share recordings of class activities with people outside the class. Recordings should be deleted at the end of the semester. Failure to follow these policies may be an Honor Code violation or lead to accountability proceedings.

Mental health and wellness. If personal or academic stress is interfering with your ability to work, Vanderbilt's Student Care Network offers a range of support. I am glad to talk about stresses related to this course and can help you connect with the Network. The Office of Student Care Coordination is the first point of contact: https://www.vanderbilt.edu/student-care-network/student-care-coordination/ or 615-343-WELL. Drop-in services: https://www.vanderbilt.edu/student-care-network/drop-in-support/

If you or someone you know needs to speak with a professional counselor immediately, the University Counseling Center offers Crisis Care Counseling. Come to the UCC to be seen by the clinician on call, or phone 615-322-2571. https://www.vanderbilt.edu/student-care-network/university-counseling-center/

Equal Opportunity and Access: https://www.vanderbilt.edu/eoa/
Student Access: https://www.vanderbilt.edu/student-access/
Title IX and student discrimination: https://www.vanderbilt.edu/title-ix/

Mandatory reporting. University faculty, many staff, and some student leaders are required to report incidents of sexual assault, sexual harassment, dating violence, domestic violence, stalking, and child abuse, as well as suspected discrimination on the basis of age, race, color, creed, religion, ancestry, national or ethnic origin, sex or gender, sexual orientation, disability, genetic information, military status, familial status, or other protected categories, to the University's Title IX Coordinator (615-343-9004), as required by University policy and state and federal law.

If you disclose an experience of interpersonal violence or child abuse to classmates with reporting obligations, or to me — in discussion or through a course assignment — your disclosure will be kept as private as possible but may not be able to be kept confidential. This includes incidents on or off campus and incidents that occurred before your time at Vanderbilt. Disclosure of thoughts of harm to yourself or others will also be shared with those responsible for the health and safety of our community.

I am willing to discuss such incidents if you wish, but only in the context of us both understanding my reporting obligations. For confidential conversations, officials in the Student Health Center, the University Counseling Center, and the Office of the Chaplain and Religious Life (when acting as clergy) can maintain confidentiality. The Project Safe Center (crisis line 615-322-7233) has limited confidentiality: staff must report incidents but can do so without identifying the person affected.

12. CS project requirement and Immersion

For undergraduates using this course to fulfil the **CS Project requirement** and/or Vanderbilt's **Immersion** requirement, participation in the **CS Immersion Showcase** is required — Thursday December 10, 2026, 5–7 p.m., Adams Atrium, Featheringill Hall **[confirm]**. Team projects are expected. Speak with me early so we can plan appropriately.

- CS Immersion: https://engineering.vanderbilt.edu/academics/undergraduate/cs-immersion/
- Immersion requirement: https://www.vanderbilt.edu/immersion/

Syllabus subject to change; changes will be announced in class and on Brightspace.